Shaheed Zulfikar Ali Bhutto Institute of Science & Technology

Total Marks: _____

Obtained Marks: _____

Date: 5-4-2026

Computer Organization and Assembly Language

Lab Report

Submitted to:	Sir.Hasan Mujtaba
Student Name:	SardarMuhammadAryan,Ali Raza,Mujeeb ur rehman
Reg No:	24108333,24108433,24108324

Binary and Hexadecimal Data Conversions in 8086 Assembly Language

Table of Contents

1. Project Overview:

The project “Binary and Hexadecimal Data Conversions in 8086 Assembly Language” demonstrates how low-level data representation is handled and converted within microprocessor programming. It focuses on developing an Assembly Language Program (ALP) that converts 8-bit binary data into its equivalent hexadecimal representation using Intel 8086 instructions.

This project applies fundamental microprocessor concepts like register operations, bit manipulation, logical instructions, and I/O handling. The main goal is to help students understand how digital systems represent and process data in different numerical bases.

In computer architecture, binary (base 2) is used for hardware-level communication, while hexadecimal (base 16) provides a human-readable shorthand for binary values. Hence, converting between these systems is a crucial operation in embedded and assembly programming.

The program reads an 8-bit binary value from an input register, processes it through logical and arithmetic instructions, and produces its hexadecimal form — which can then be displayed or sent to an output port.

This project not only strengthens understanding of data conversion logic, but also illustrates how 8086 registers (AL, BL, CL, DL) and logical operations (AND, OR, SHL, SHR) are used in real applications.

2. Scope of the Project:

The project covers all necessary steps to convert binary data into hexadecimal using 8086 Assembly Language and to display or output the converted result.

Main Features Covered:
• Binary-to-Hexadecimal Conversion: The system accepts an 8-bit binary input (e.g., from a port or memory). Divides the byte into two 4-bit nibbles. Each nibble is converted into its corresponding hexadecimal digit (0–9, A–F).
• Hexadecimal-to-Binary Conversion (Optional): The program can also demonstrate reverse conversion by mapping each hex digit back to its binary form.
• Use of Registers and Instructions: Implements MOV, SHR, AND, CMP, ADD, and conditional jumps for logic. Makes use of register manipulation and bitwise operations to process data.
• I/O Port Handling: The program can be integrated with IN and OUT instructions to read binary input from a port and output the hexadecimal result.
• Educational and Practical Value: Reinforces understanding of how processors handle data and provides practical application of number system conversion logic.

Limitations:

• Data is processed in 8-bit only (no word or double-word support).
• Works in a simulation environment (emu8086), not on actual hardware.
• The program does not include permanent data storage or GUI.

3. Project Modules:

3.1. Input Module

Purpose: Reads an 8-bit binary number from user input, register, or port.
Implementation: Uses MOV AL, <data> or IN AL, port instruction to load binary data.
Example: MOV AL, 10101100b
Function: Stores the binary data in the accumulator (AL register) for processing.

3.2. Conversion Logic Module

Purpose: Handles the main binary-to-hexadecimal conversion.
Implementation Steps:
• Copy AL → BL for processing.
• Shift right by 4 bits (SHR BL, 4) to isolate the upper nibble.
• Convert the upper nibble to ASCII (add 30h and adjust above 39h).
• Mask lower nibble (AND AL, 0Fh) and repeat conversion steps.
• Store both results for display.
Output Example: Input: 10101100b → Output: "AC"

3.3. Output Module

Purpose: Displays or sends the converted hexadecimal data.
Implementation: Uses registers DL and DH to store ASCII characters and INT 21h for displaying characters.

3.4. Execution and Debug Module

Purpose: Test and verify the logic using emulator tools like emu8086.
Implementation: Step-by-step execution using emulator monitor to verify registers and flags.

4. Tools and Technologies:

• emu8086 Microprocessor Emulator – To write, compile, and execute 8086 assembly code in a simulated environment.
• 8086 Assembly Language – To perform bitwise operations and handle registers directly.
• Intel 8086 Architecture – Underlying system for instruction execution and register handling.
• MS-DOS Interrupts (INT 21h) – Used to display output and handle basic I/O functions.

5. Deliverables:

1. Assembly Source Code (.asm file) – Well-commented, structured, and runnable in emu8086.
2. Executable Simulation Output – Showing binary input and hexadecimal output.
3. Documentation Report – Explaining logic, registers, and step-by-step working.
4. Presentation Slides – A short presentation explaining the process, logic, and code.

6. Challenges Faced:

1. Handling ASCII Conversion – Managing numeric to ASCII conversions using 30h and 07h adjustments.
2. Nibble Separation Errors – Incorrect shifting or masking initially caused wrong hex outputs.
3. Limited Debug Visibility – Emulator flags and register updates needed careful tracking.
4. I/O Instruction Restrictions – IN and OUT commands required valid port addresses.

7. Future Improvements:

- Add Hex-to-Binary reverse conversion.
- Include 16-bit conversions.
- Integrate I/O ports for real hardware simulation.
- Display results on virtual 7-segment LED interface.
- Add error handling for invalid inputs.

8. Code Explanation and Output:

8086 Assembly Code Example:

```
; Binary to Hexadecimal Conversion Program
; Written for emu8086 simulator

MOV AL, 10101100b
MOV BL, AL
MOV CL, 4
SHR BL, CL
ADD BL, 30h
CMP BL, 39h
JBE NEXT1
ADD BL, 07h
NEXT1:
MOV DL, BL
AND AL, 0Fh
ADD AL, 30h
CMP AL, 39h
JBE NEXT2
ADD AL, 07h
NEXT2:
MOV DH, AL
MOV AH, 02h
MOV DL, DL
INT 21h
MOV DL, DH
INT 21h
```

HLT

Output:

Binary Input → 10101100b
Result → ACh

9. Conclusion:

The project "Binary and Hexadecimal Data Conversions in 8086 Assembly Language" demonstrates how number system conversions can be implemented directly at the hardware instruction level. It deepens understanding of register-based data manipulation, logical operations, and microprocessor-level programming.

By performing conversions manually using MOV, AND, SHR, and ADD instructions, this project helps learners visualize how processors interpret, process, and display numeric data. It also shows the importance of ASCII mapping, bit masking, and conditional branching in real-time embedded computation.

Overall, this project bridges theoretical number systems and their practical implementation in microprocessor programming, serving as a valuable learning experience for students.